

PYTHON LIBS FOR AI AGENTS.

8 libraries that fix real problems – not theoretical ones. Each one earned its place by preventing production incidents, cutting costs, or saving hours of debugging.

01

LiteLLM

Switch providers by changing a string

02

Instructor

Guaranteed structured LLM output

03

Tenacity

Retry logic that actually works

04

Logfire

Observability beyond "Processing complete"

05

Diskcache

Persistent cache, no Redis required

06

Tiktoken

Exact token counts before you send

07

Rich

Debug output you can actually read

08

Watchfiles

Auto-reload on every file save

▲ LIBRARY 01

LiteLLM

PIP INSTALL LITELLM

One interface. **100+ models**. Switch providers by changing a string.

BEFORE VS AFTER

```
from litellm import completion

# GPT-4
completion(model="gpt-4", messages=[...])

# Claude – same function, different string
completion(model="claude-3-opus-20240229", messages=[...])

# Local Llama via Ollama – still same function
completion(model="ollama/llama2", messages=[...])
```

REAL COST Same workload across GPT-4, Claude Sonnet, Llama 3 70B – **\$847/month difference**. Without LiteLLM, testing that would've taken a week of refactoring.

USE FOR Multi-model testing, provider fallbacks, cost optimization

PRO TIP Set up **automatic fallbacks**. When OpenAI goes down, your agent switches to Anthropic without you waking up.

Instructor

PIP INSTALL INSTRUCTOR

LLMs lie about JSON. Instructor guarantees valid output.

STRUCTURED EXTRACTION WITH VALIDATION

```
import instructor
from pydantic import BaseModel
from openai import OpenAI

client = instructor.from_openai(OpenAI())

class UserInfo(BaseModel):
    name: str
    age: int

user = client.chat.completions.create(
    model="gpt-4",
    messages=[{"role": "user", "content": "Extract: John is 25"}],
    response_model=UserInfo,
)
```

REAL IMPACT Replaced a **150-line** `parse_llm_json()` function with twelve edge cases. Zero parsing errors in four months since switching.

USE FOR Structured extraction, tool call parsing, data pipelines

INSIGHT If extraction fails, Instructor shows the model what went wrong and **retries automatically**. Same philosophy as Pydantic AI – pairs perfectly with it.

Tenacity

PIP INSTALL TENACITY

APIs will fail at 3am. Tenacity catches it before you wake up.

EXPONENTIAL BACKOFF IN ONE DECORATOR

```
from tenacity import (
    retry, stop_after_attempt, wait_exponential
)

@retry(
    stop=stop_after_attempt(3),
    wait=wait_exponential(multiplier=1, min=4, max=10)
)
def call_flaky_api():
    return external_service.get_data()
```

REAL INCIDENT 47 Slack alerts on a Sunday because a third-party API returned 503s for 20 minutes. The "robust" error handling just logged and moved on. **One decorator would've fixed it.**

USE FOR Every external API call. Every single one.

WARNING Do not retry on **4xx errors**. You will get rate limited faster. Only retry on 5xx and network failures.

Logfire

PIP INSTALL LOGFIRE

"Processing complete" is not debugging. **Logfire shows exactly what happened.**

STRUCTURED OBSERVABILITY

```
import logfire

logfire.configure()

@logfire.instrument("process_user_request")
def handle_request(user_input: str):
    logfire.info("Received input", input=user_input)
    result = agent.run(user_input)
    logfire.info("Agent complete",
                 result=result, tokens_used=result.tokens)
    return result
```

3 HOURS VS 5 MINUTES Spent 3 hours debugging why an agent kept recommending the wrong product. The retrieval step was returning **stale data**. Logfire would've caught it in 5 minutes.

USE FOR Production debugging, token tracking, latency monitoring

PRO TIP Integrates with Pydantic – structured data gets logged with **full type information** automatically.

Diskcache

PIP INSTALL DISKCACHE

Persistent cache. Survives restarts. **No Redis server to maintain.**

```
from diskcache import Cache
cache = Cache('./agent_cache')

@cache.memoize(expire=3600)
def expensive_embedding(text: str):
    return openai.embeddings.create(input=text, model="text-embedding-3-small")
```

REAL SAVING Added to an embedding pipeline making **10K+ calls/day**. API costs dropped 34% the first week.

Tiktoken

PIP INSTALL TIKTOKEN

Count tokens exactly **before you send** — not after the crash.

```
import tiktoken
enc = tiktoken.encoding_for_model("gpt-4")

def fits_in_context(messages: list, max_tokens=8000) -> bool:
    total = sum(len(enc.encode(m["content"])) for m in messages)
    return total < max_tokens
```

WARNING Character-based estimates are wrong **30% of the time** on real data — especially with code and special characters.

Rich

PIP INSTALL RICH

Stop squinting at 500-line JSON blobs. **Debug output you can actually read.**

TABLES, JSON, PROGRESS BARS – ONE IMPORT

```
from rich.console import Console
from rich.table import Table
from rich import print_json

console = Console()

# Pretty-print any JSON with syntax highlighting
print_json(data=agent_response)

# Build a readable table of agent actions
table = Table(title="Agent Actions")
table.add_column("Step")
table.add_column("Tool")
table.add_column("Result")
for step in agent.history:
    table.add_row(str(step.num), step.tool, step.result[:50])
console.print(table)
```

USE FOR Development debugging, demo outputs, CLI interfaces

ONE-LINER UPGRADE `from rich import print` replaces the built-in `print`. One import, instant syntax highlighting and formatting for everything.

Watchfiles

PIP INSTALL WATCHFILES

Change a prompt. Save the file. **Agent restarts instantly. No manual intervention.**

AUTO-RELOAD ON EVERY SAVE

```
from watchfiles import run_process

def main():
    from my_agent import Agent
    agent = Agent()
    agent.run_interactive()

if __name__ == "__main__":
    # Watches ./ for any changes
    run_process("./", target=main)
```

REAL NUMBERS Tracked workflow for one day: **40+ manual restarts**. Each took 8 seconds. Over **5 minutes of waiting per day**, for weeks. Watchfiles eliminated all of it.

USE FOR Local development, prompt iteration, rapid prototyping

SMART RELOADS Modify a **prompt file – instant reload**. Save a log file – no reload. It knows which changes matter.

▲ THE FULL STACK

YOUR AGENT TOOLKIT.

Each library solves exactly one problem – and solves it completely.
Stack them together and your agent is production-ready.

LiteLLM

Provider-agnostic. One interface for all models. Build-in fallbacks.

Instructor

Guaranteed structured output from any LLM. Pydantic-native.

Tenacity

Exponential backoff on every external call. One decorator.

Logfire

Structured traces. See exactly what the agent did and why it went wrong.

Diskcache

Persistent memoization without Redis. Cuts API costs on repeated calls.

Tiktoken

Exact token counts. Prevent context overflow before it crashes the agent.

Rich

Readable debug output. Tables, highlighting, and progress bars in one line.

Watchfiles

Auto-reload on save. Eliminate manual restarts during development.